

Coarse-Grained Parallel Genetic Algorithm for Solving the Timetable Problem

Shisanu Tongchim

Department of Computer Engineering, Faculty of Engineering,
Chulalongkorn University, Bangkok 10330, Thailand

Tel: (662) 218-6983 Fax: (662) 218-6955

g41stc@cp.eng.chula.ac.th

Abstract

This work studies the application of Genetic Algorithm (GA) to solve the timetable problem (TTP), a problem that is a highly constrained problem. However, the major disadvantage of GA is that the algorithm uses a tremendous amount of processing time. This paper proposes a method that can reduce the processing time by using Parallel Genetic Algorithm (PGA) on the Beowulf-style cluster computer. The experimental results show that substantial speedup of GA can be acquired with small communication overhead.

1. Introduction

The timetable problem (TTP) – a particular form of the general scheduling problem – is a complex problem because of many conflicting constraints [13]. As a way to solve TTP, several pieces of research [5,6,7,8,14] apply genetic algorithm (GA) to find an optimal solution, which satisfies nontrivial constraints. **GA can be used as a technique for solving complex problems which have very large search space [3].** The major disadvantage of GA is that the algorithm uses a very large amount of processing time. A parallel algorithm approach can be applied to the classical GA for reducing processing time. Parallel Genetic Algorithm (PGA) can be classified into three different models: *Master-slave PGA*, *Coarse-grained PGA* and *Fine-grained PGA*.

This paper presents a method to solve the timetable problem based on Coarse-grained Parallel GA, which is suitable for the Beowulf-style cluster computer – a low-cost parallel computer composed of cheap PCs and a low speed network [1]. The Beowulf-style cluster computer is popular because of its low price, availability of free software (like MPI and PVM) [10], and its ease of implementation. Other work on applying PGA to timetable problems [2], uses **Master-Slave parallelization method implemented on a commercial shared memory multiprocessor. However Master-Slave parallelization is machine dependent [12].** A shared memory multiprocessor computer is not easily available when compared with a clustered computer, which consists of network of workstations.

This paper is organized as follows: The next section is a problem formulation and Section 3 describes a problem representation in GA. Section 4 shows a parallel solution and Section 5 gives the experimental results. Finally, Section 6 provides the conclusions of this project.

2. The Timetable Problem

The timetable problem can be categorized into various types (e.g., rail timetable, examination timetable, etc.) by considering different specific constraints, processes, resources. The particular instance chosen for implementation is the lecture timetable. The lecture timetable

problem is to organize **Subject, Teacher, Room, and Period** in order to satisfy the set of constraints.

The problem formulation is as follows:

Set of subjects $S = \{s_1, s_2, \dots, s_n\}$

Set of teachers $T = \{t_1, t_2, \dots, t_m\}$

Set of rooms $R = \{r_1, r_2, \dots, r_o\}$

Set of periods $P = \{p_1, p_2, \dots, p_k\}$

Tuple [2], a combination (a,b,c,d) in which $a \in \mathbf{S}$, $b \in \mathbf{T}$, $c \in \mathbf{R}$, $d \in \mathbf{P}$, can be stated as “teacher b teaches subject a in room c during period d ”. The relation of subjects and teachers are fixed in all tuples. The lecture timetable problem can be interpreted as the scheduled rooms and periods in tuples and the solution of this problem is a set of tuples which satisfy all constraints. The set of constraints in the experiment are defined as follows:

1. **Resource conflict:** The teacher and the room are resources that must not occur in the same period.
2. **Option:** Some courses cannot be scheduled in the same period. Students may wish to take both courses. A constraint is written in the combination of two subject numbers. For example, a rule (5,26) represents “subject 5 and subject 26 cannot be in the same period”.
3. **Exclusion:** Some courses should not be scheduled in specific period or specific room. A time exclusion constraint is written in the combination of a subject number and a period number. For example, a rule (12,5) represents “subject 12 cannot be scheduled in the period 5”. A room exclusion constraint is written in the combination of a subject number and a room number. For example, a rule (20,3) represents “subject 20 cannot be scheduled in the room 3”.
4. **Juxtaposition:** A lecture session should be placed earlier than a laboratory session. A constraint is written in the combination of two subject numbers. One subject is a lecture session and one subject is a laboratory session. For example, a rule (4,5) represents “subject 4 should be placed earlier than subject 5 in the timetable”.

Each constraint can be assigned a different weight, reflecting constraint significance.

3. Genetic Representation

Scheduling performs with rooms and periods particularly because the relation of subjects and teachers in tuples are not changed, hence a chromosome keeps only rooms and periods. A chromosome is implemented by two-dimension array which has area $2n$, where n is the number of all subjects, divided into two equal groups. Rooms are kept in the first group and periods are kept in the second group (see figure 1). For example, the period number in the second position is 23 and the room number in the same position is 5, then subject 2 is scheduled to occur at time 23 in room 5. Other work [2], uses alternative chromosome representation where each position in a chromosome represents the period and the set of classes which are scheduled in the same period take place. This representation allows good scheduling to be

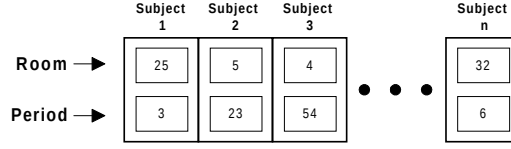


Figure 1: Chromosome representation

preserved between generations; however, this mapping can cause *label replacement* problem – the crossover operation produces invalid children.

The genetic operators are defined as follows:

- **Selection:** The selection scheme uses *Tournament selection*. The selection pressure of this method varies with the tournament size – the more competitors, the higher selection pressure. In the experiment, the tournament size is three.
- **Reproduction:** Every individual has a specific chance to be selected for reproduction according to the assigned fitness value.
- **Crossover:** Two populations are selected. One point crossover is applied two times, between the room attribute and the period attribute with independent crossover sites. See Figure 2.
- **Mutation:** One population is selected. Mutation operator swaps two values at random mutation positions. This operator applies with both attribute at independent mutation sites. See Figure 2.

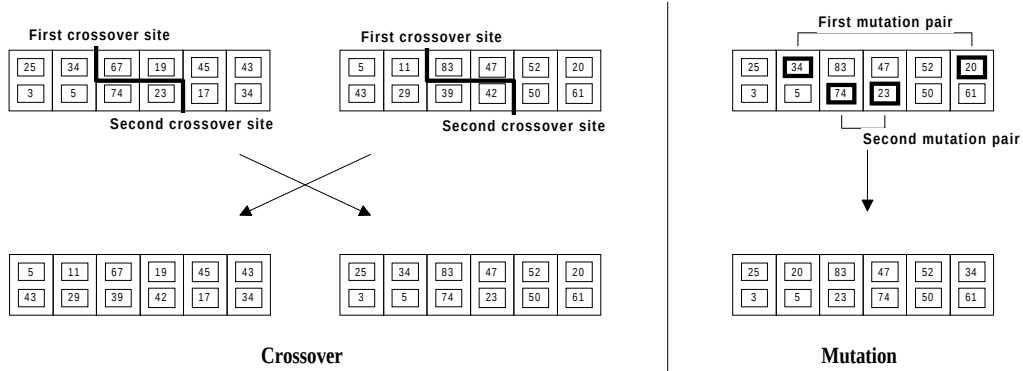


Figure 2: Genetic operator

The fitness function is a sum of penalties.

$$f(t) = \sum_{i=1}^n \sum_{j \in c} W_j p_j(g_i) \quad (1)$$

t is a timetable as a collection of tuples $\{g_1, g_2, \dots, g_n\}$

n is the number of tuples

C is the set of constraints

W_j is the weight for constraint j

$p_j(g_i)$ is the function that return 1 if tuple g_i violates constrain j , and 0 for otherwise

The algorithm must minimize the fitness function and the fitness value of the optimum solution of zero.

4. Parallel Genetic Algorithm (PGA)

GA has the ability to be parallelized because an algorithm work with a set of population, not only an individual. The evolution of an individual is separated from each other. The concept of PGA is to divide the task of the classical GA and distributes on different processors. There are three main types of PGA:

- **Master-Slave PGA** This model uses a single global population and the fitness evaluation is done on different processors. Furthermore, genetic operations may also be done in parallel. The nature of GA is not changed because an algorithm still works with the whole population. A global population is suitable for a shared memory computer.
- **Coarse-grained PGA** The population is divided into a few large subpopulations. Each of these subpopulations are maintained by different processors and some selected individuals are exchangeable via a migration operator. The model is known as Island model or distributed PGA and subpopulation called *deme* [9]. Island model is a popular and effective parallel genetic algorithm [4] and also reduces probability of premature convergence [11] – finding the local instead of the global optimum.
- **Fine-grained PGA** The population is separated into a large number of very small subpopulations, which are maintained by different processors. The subpopulation may be only an individual. This model is suitable for massively parallel architectures – machines consisting of a huge number of basic processors and connected with a specific high speed topology. The computer structure limits an interaction between individuals. This model is machine dependent like Master-Slave PGA.

The model selection is based on three considerations: affordability, the ease of implementation and programming effort. Coarse-grained PGA is selected because this model is easily implemented by a clustered computer and using public domain software. A little effort is needed for converting classical GA to this model by adding migration operator. A clustered computer is composed of:

- 12 workstations with Intel Pentium 133 MHz with 16 Mbytes main memory and 1 Ethernet (10 baseT) per processor
- 1 Ethernet Hub
- 1 workstation with Intel 486 66 MHz with 24 Mbytes main memory and 2 Ethernet (10 baseT)

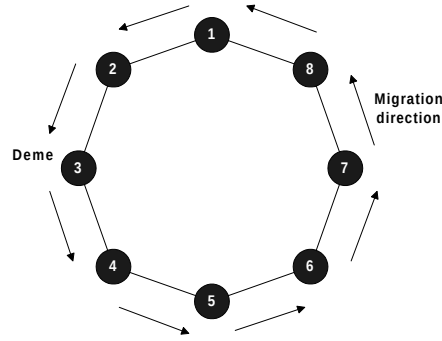


Figure 3: Topology

Processing nodes are implemented on 12 Pentium workstations and use a 486 workstation as a gateway to separate network traffic from outside. All workstations use Linux as an operating system and MPI as message passing library. The connection is ring topology (See Figure 3). By synchronizing migration, the communication is separated in two steps. First, the nodes with odd numbers send the population to another even number. Then, the nodes with even numbers send the population to another odd number in the same direction.

5. Computational Result

Table 1 shows the algorithm parameters. The problem parameters and the number of constraints are shown in table 2. The set of constraints are randomly generated as an input file. The node number varies from 1 to 12 which uses the same set of constraints and each experiment is run 20 times.

Table 1: GA parameters

Population size per node	500
Crossover probability	0.6
Mutation probability	0.01
Migration size	10%
Maximum generation	1000
Tournament size	3

The efficiency of Parallel GA is measured in terms of computational effort, defined as the number of individual that must be processed to get solution, compared to the serial algorithm [15]. The computational effort is redefined for parallel performance measurement because all demes evolve simultaneously on different processors and communication overhead is so small. Hence the computational effort per number of nodes is defined for parallel performance evaluation. The experimental results in figure 4 show that parallel algorithm can reduce computational effort per node when compares with serial GA that computational effort per node is 4921500.

Table 2: Problem parameters

Timeslot	55
Subject	500
Teacher	300
Room	200
Option constraint	800
Time exclusion constraint	400
Room exclusion constraint	400
Juxtaposition constraint	100

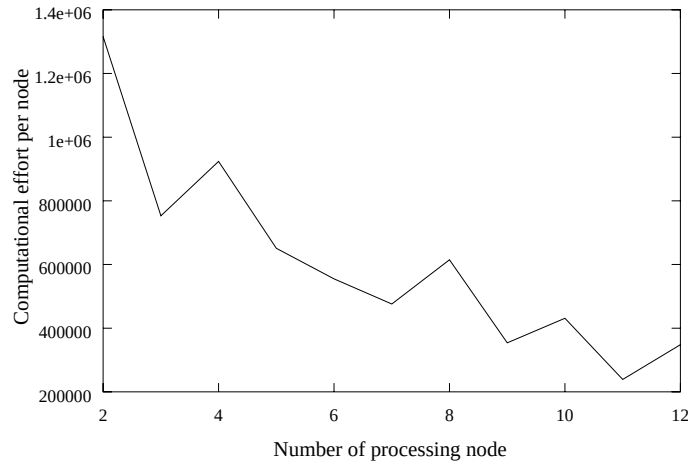


Figure 4: Computational effort per node

It shows that the computational effort per number of nodes decreases towards the increment of number of processors.

The overhead due to the communication among the processors is small and increases slightly when there are increases in the number of nodes (see figure 5). The increment of message passing overhead when there are increases in the number of nodes is caused by network architecture. Ethernet hub uses the CSMA/CD protocol for operation [16]. All nodes connect to the same collision domain and hence transmissions cannot take place in parallel. When there are increases in the number of nodes, it also increases packet collision probability.

From the convergence graph in figure 6. The graph shows that increase number of processing nodes does not help much on improving convergence. However the difference between the results with parallel and serial algorithm is even more evident. This can be explained by investigating the PGA operation. Coarse-grained PGA maintains multiple subpopulations which evolve independently on different processors. Each node uses a different random seed, hence this allows each subpopulation to explore different parts of the search space. By increasing the number of subpopulations, it also increases the probability in finding the optimum solution.

6. Summary and conclusions

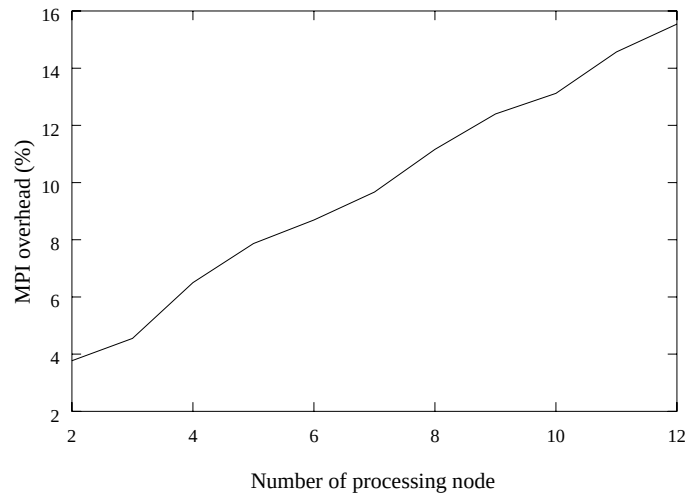


Figure 5: MPI overhead

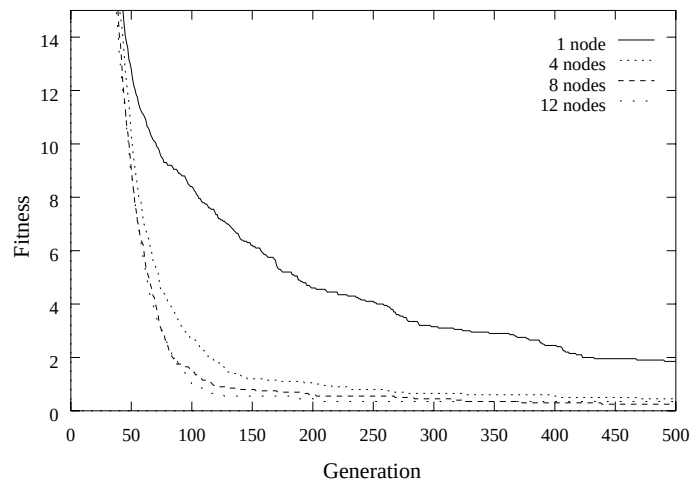


Figure 6: Convergence graph

This paper has proposed a GA based technique for solving the timetable problem. A parallel algorithm approach can be introduced to basic GA for reducing processing time and improving search performance. The coarse-grained PGA implementation on the Beowulf-style cluster computer is selected with three criteria: affordability, the ease of implementation and programming effort. Coarse-grained PGA is suitable for a clustered computer, which is easily implemented from cheap PCs connected through a low cost network and using public domain software. A little effort is needed for converting classical GA to coarse-grained PGA. Each processor performs simple GA and periodically exchanges some population by migration operators.

The experiment compares the performance of classical GA and coarse-grained PGA with varying a number of processing nodes. Performance analysis of the parallel algorithm is measured in terms of computational effort per number of nodes, and the convergence graph compared to the serial algorithm.

The difference between the empirical efficiency of parallel and serial algorithm is clear. There is an improvement with large number of processors. Furthermore, the overhead due to the communication among the processors is small and slightly rises towards an increment of processors. In the case of ten processors, the overhead is about 13% of the total computational time.

References

- [1] D. J. Becker, T. Sterling, D. Savarese, J. E. Dorband, U. A. Ranawak, C. V. Packer, "BE-OWULF: A PARALLEL WORKSTATION FOR SCIENTIFIC COMPUTATION", Proceedings, International Conference on Parallel Processing, 1995
- [2] D. A. Abramson, J. Abela, "A Parallel Genetic Algorithm for Solving the School Timetabling Problem", IJCAI workshop on Parallel Processing in AI, Sydney, August 91. Also appearing in 15 Australian Computer Science Conference, Hobart, Feb 1992, pp. 1 - 11.
- [3] D. E. Goldberg, "Genetic Algorithms in Search, Optimization & Machine Learning", Addison Wesley, Reading, 1989
- [4] D. Whitley, S. Rana and R. B. Heckendorn, "Island Model Genetic Algorithms and Linearly Separable Problems", Proceedings of the AISB Workshop on Evolutionary Computation, 1997
- [5] M. Dorigo, V. Maniezzo, A. Coloni, "Genetic Algorithms and highly constrained problems: the time- table case", Proceedings of the First International Workshop on Parallel Problem Solving from Nature, Dortmund, Germany, Lecture Notes in Computer Science 496, Springer-Verlag, pp. 55-59.
- [6] D. Corne, H-L Fang, C. Mellish, "Solving the Module Exam Scheduling Problem with Genetic Algorithms", In Proceedings of the Sixth International Conference on Industrial and Engineering Applications of Artificial Intelligence and Expert Systems, Gordon and Breach Science Publishers, Chung, Lovegrove, Ali (eds), 1993, pp. 370-373.
- [7] E.K. Burke, D.G. Elliman, R.F. Weare, "A Genetic Algorithm Based University Timetabling System", East-West Conference on Computer Technologies in Education, Crimea, Ukraine,

1994, pp. 35-40.

[8] P. Ross, D. Corne, H-L Fang, "Successful Lecture Timetabling with Evolutionary Algorithms", Appears in Workshop Notes, ECAI'94 Workshop W17: Applied Genetic and other Evolutionary Algorithms, organised by A.E.Eiben, B.Manderick, Zs.Ruttkey.

[9] D. Andre, J. R. Koza, "Parallel genetic programming on a network of transputers", In Rosca, Justinian (editor), Proceedings of the Workshop on Genetic Programming: From Theory to Real-World Applications, University of Rochester, National Resource Laboratory for the Study of Brain and Behavior, Technical Report 95-2, June 1995, pp. 111 - 120.

[10] E. Cantú-Paz (1998), "A survey of parallel genetic algorithms", *Calculateurs Paralleles*. Vol. 10, No. 2. Paris: Hermes.

[11] S-C Lin, W.F. Punch and E.D. Goodman, "Coarse- grain Genetic Algorithms, Categorization and New Approaches", Sixth IEEE Parallel and Distributed Processing Oct 1994, pp. 28-37.

[12] A. L. Corcoran and R. L. Wainwright, "A Parallel Island Model Genetic Algorithm for the Multiprocessor Scheduling Problem", Proceedings of the 1994 ACM/SIGAPP Symposium on Applied Computing, ACM Press, March 6-8, 1994, pp. 483-487.

[13] E. Burke and P. Ross, "Practice and Theory of Automated Timetabling", Lecture Notes in Computer Science 1153, Springer-Verlag, 1996.

[14] W. Salwach, "Genetic Algorithms in solving Constraint Satisfaction Problems: The Timetable Case", *Badania Operacyjne i Decyzje*, 2/1997.

[15] J. R. Koza, "Genetic Programming II: Automatic discovery of reusable programs", MIT Press, Cambridge, MA, 1994, pp. 99-106.

[16] F. Halsall, "Data Communications, Computer Networks and Open Systems", Addison-Wesley, Fourth edition, 1995.